

CENTRO UNIVERSITÁRIO UNIRUY | WYDEN
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SOFTWARE
DISCIPLINA DE ESTRUTURA DE DADOS

ERIC ASSIS

**ATIVIDADE E DESAFIO: APLICAÇÃO DE ESTRUTURAS DE DADOS NO
MUNDO REAL**
ANÁLISE DE PILHAS, ÁRVORES, TABELAS HASH, B-TREES E GRAFOS

SALVADOR – BA
2026

ERIC ASSIS

**ATIVIDADE E DESAFIO: APLICAÇÃO DE ESTRUTURAS DE DADOS NO
MUNDO REAL**

ANÁLISE DE PILHAS, ÁRVORES, TABELAS HASH, B-TREES E GRAFOS

Atividade de pesquisa e resolução de desafio
apresentada à disciplina de Estrutura de Dados do
Centro Universitário UniRuy Wyden, como
requisito de avaliação.

Docente: Prof. Heleno Cardoso

SALVADOR – BA
2026

1. ONDE EXISTE UMA PILHA NO WINDOWS?

No sistema operacional Microsoft Windows, a estrutura linear do tipo Pilha (LIFO, Last In, First Out) opera diretamente em pontos essenciais do sistema:

- Pilha de Execução (Call Stack): Cada thread criada no Windows possui uma pilha dedicada na memória RAM. Ao chamar uma função, os parâmetros, as variáveis locais e o endereço de retorno são empilhados através dos registradores ESP (32 bits) ou RSP (64 bits). Ao término da função, o processador executa a instrução RET, desempilha os dados e retorna a execução exatamente à linha do código chamador.
- Mecanismo de Desfazer (Ctrl + Z): No Windows Explorer e em aplicações como o Bloco de Notas e Word, as ações do usuário são empilhadas sequencialmente. Ao pressionar o atalho Ctrl + Z, a ação mais recente do topo da pilha é desempilhada e revertida.

2. ONDE UMA ÁRVORE APARECE NO LINUX?

O sistema operacional Linux utiliza estruturas em Árvore para organizar o sistema de arquivos e balancear a execução de tarefas no processador:

- Sistema de Arquivos Virtual (VFS): Toda a hierarquia de arquivos do Linux é estruturada como uma árvore que parte do diretório raiz (/). Pastas e arquivos são nós (estruturas dentry) conectados aos nós pais. Na resolução de um caminho como /usr/bin/python3, o VFS percorre a árvore descendo nó por nó a partir da raiz.
- Escalonador de Processos (CFS): O Completely Fair Scheduler organiza os processos prontos para execução em uma Árvore Rubro-Negra (Red-Black Tree). Os nós são indexados pelo tempo virtual de CPU consumido (vruntime). O kernel sempre seleciona o nó mais à esquerda da árvore para rodar primeiro, garantindo divisão justa de processamento e busca em tempo logarítmico $O(\log n)$.

3. COMO O SPOTIFY ORGANIZA MÚSICAS?

O Spotify combina diferentes estruturas de dados especializadas para garantir reprodução ágil e gerenciamento dinâmico de faixas:

- Listas Duplamente Encadeadas: A playlist ativa funciona como uma lista duplamente encadeada, na qual cada música mantém ponteiros para a faixa anterior (prev) e para a próxima (next). Isso possibilita avançar e voltar de faixa instantaneamente, bem como reordenar faixas por arrastar e soltar em tempo constante $O(1)$, ajustando apenas os ponteiros dos nós afetados.
- Fila com Prioridade: A opção 'Adicionar à Fila' atua como uma fila com prioridade (FIFO). As faixas adicionadas manualmente são consumidas prioritariamente pelo reprodutor antes de seguir o fluxo normal da playlist.
- Tabelas Hash: Cada faixa possui um identificador único em Base62 (Track URI). O Spotify consulta tabelas hash em memória (e caches como Redis) para recuperar instantaneamente os metadados da música (título, artista, álbum, link do áudio) em tempo médio constante $O(1)$, sem necessidade de varredura linear.

4. POR QUE UM BANCO DE DADOS USA B-TREE?

Sistemas Gerenciadores de Bancos de Dados relacionais (como PostgreSQL, MySQL e Oracle) utilizam a B-Tree e a B+ Tree em seus índices pelos seguintes fatores determinantes:

- Redução de I/O em disco: O acesso ao disco (HDD ou SSD) é ordens de grandeza mais lento que a memória RAM, transferindo blocos/páginas fixas (comumente 4 KB ou 8 KB). Enquanto uma árvore binária possui apenas 2 filhos por nó e exigiria dezenas de níveis para milhões de registros, a B-Tree possui um alto fator de ramificação (cada nó armazena centenas de chaves e ponteiros, preenchendo uma página de disco inteira). Isso reduz a altura da árvore para apenas 3 ou 4 níveis, localizando qualquer registro com pouquíssimas leituras de disco.
- Varredura por Intervalos (Range Queries na B+ Tree): Na B+ Tree, todos os dados residem nos nós-folha, e esses nós são encadeados horizontalmente em uma lista ligada. Em consultas com operadores como BETWEEN, >, < e ORDER BY, o banco localiza a chave inicial em tempo $O(\log n)$ e apenas percorre a lista de folhas sequencialmente, sem retornar aos nós superiores.

5. DESAFIO: UMA SITUAÇÃO REAL ONDE UMA ESTRUTURA DE DADOS É FUNDAMENTAL

Aplicação Selecionada: Sistemas de Navegação e Roteamento GPS (Google Maps, Waze) utilizando Grafos e Filas de Prioridade (Min-Heap).

- **Modelagem em Grafo:** A malha viária é representada como um grafo ponderado e direcionado. Os cruzamentos e esquinas são os vértices (nós) e os trechos de ruas são as arestas. Cada aresta possui um peso (distância em metros ou tempo estimado com base no trânsito em tempo real) e uma direção (mão única ou mão dupla).
- **Cálculo de Menor Caminho:** Para determinar o trajeto mais rápido entre a origem e o destino, o sistema executa algoritmos como Dijkstra ou A* (A-Star).
- **Papel da Fila de Prioridade (Min-Heap):** Durante a busca, os cruzamentos vizinhos a serem explorados ficam armazenados em um Min-Heap. Isso permite extrair o vértice de menor custo acumulado em tempo logarítmico, descartando caminhos lentos e calculando a melhor rota em frações de segundo.

REFERÊNCIAS BIBLIOGRÁFICAS

CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.

LOVE, Robert. **Linux Kernel Development**. 3. ed. Upper Saddle River: Addison-Wesley, 2010.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015.

TANENBAUM, Andrew S.; BOS, Herbert. **Sistemas Operacionais Modernos**. 4. ed. São Paulo: Pearson, 2016.

ZIVIANI, Nivio. **Projeto de Algoritmos: Com Implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2011.